OCR de 2014 en una sola PC del laboratorio: directorios, comandos y qué esperar

5 de noviembre de 2026

Resumen

Guía para ejecutar el OCR de los escaneos de 2014 en una única PC del laboratorio, sin privilegios de administrador y sin internet. Todo lo necesario viaja en el USB: el intérprete de Python, Tesseract con el idioma español y poppler son portátiles y se ejecutan desde allí. El documento fija el directorio de trabajo, los comandos en orden, el paralelismo para equipos con muchos núcleos, cómo comprobar el resultado y qué partes no se han probado todavía.

Índice

1. Punto de partida **1**

2. Qué hay en el USB **1**

3. Procedimiento **2**
3.1. Directorio donde entrar 2
3.2. Paso 1: prueba con 50 PDFs 2
3.3. Paso 2: OCR de todo 2014 2
3.4. Si se corta 3
3.5. Paso 3: convertir a markdown 3

4. Paralelismo **3**

5. Cuánto tardará **4**

6. Comprobar el resultado **4**

7. Después del OCR **4**

8. Cambios en el código **5**

9. Si el laboratorio no deja ejecutar los `.cmd` **5**

10. Qué no se ha probado **6**

1. Punto de partida

Los PDFs de 2014 están descargados completos y casi todos son fotografías de la resolución sin capa de texto, así que `pdftotext` no devuelve nada y no pueden convertirse a markdown hasta pasarles OCR.

Concepto	Cantidad
PDFs de 2014 en el manifiesto y en el USB	11 473
Con texto de origen (ya convertidos a markdown)	70
Sin texto, que necesitan OCR	11 403
Con OCR hecho durante las pruebas del 4–5 de nov.	65
Pendientes aproximados	**11 338**

El manifiesto (`manifiesto-pdfs.csv`) y el Excel limpio (`todo-corte-suprema-completo-limpio.xlsx`) tienen 209 180 filas cada uno y coinciden con los PDFs de 2014 que hay en el USB. Se copiaron al USB y su copia es idéntica a la original.

2. Qué hay en el USB

En esta guía `X:` es la letra que Windows asigne al USB en esa PC; se ve en el Explorador de archivos.

```
X:\
|-- ocr-offline\ <-- directorio de trabajo
| |-- parte-sola.cmd todo 2014 en 3 tandas (el que se usa)
| |-- ocr-lab.cmd OCR directo sobre una carpeta, y -Convertir
| |-- ocr-parte.ps1, ocr-lab.ps1 los scripts que lanzan los .cmd
| |-- COMANDOS.txt esta misma lista, en texto plano
| |-- portable\ Python, Tesseract (spa), poppler
| |-- wheels\ paquetes de Python, para reconstruir
| `-- instaladores\ copias de los instaladores originales
|-- SCRAP\testeo\ ocr.py, a_markdown.py y el resto del codigo
`-- corpus-corte-suprema\
    |-- pdfs\<especialidad>\2014\ los PDFs
    |-- markdown\ markdown ya generado
    |-- manifiesto-pdfs.csv
    `-- todo-corte-suprema-completo-limpio.xlsx
```

No se instala nada. Los `.cmd` llaman a PowerShell con `-ExecutionPolicy Bypass` solo para ese proceso, y los scripts añaden Tesseract y poppler al `PATH` únicamente de esa ventana. Al cerrarla no queda nada cambiado en la PC.

3. Procedimiento

3.1. Directorio donde entrar

Siempre el mismo:

```
X:\ocr-offline
```

Forma rápida: abrir esa carpeta en el Explorador, hacer clic en la barra de direcciones, escribir `cmd` y pulsar Enter. Por consola: `X:` y luego `cd \ocr-offline`.

3.2. Paso 1: prueba con 50 PDFs

Se hace una vez, para comprobar que la PC deja ejecutar todo desde el USB y para medir la velocidad real de esa máquina. Se hace directamente sobre el USB y sobre una sola especialidad pequeña (civil, 820 PDFs), sin copiar nada al disco local:

```
ocr-lab.cmd -Limite 50 -Especialidad civil
```

Debe terminar con `OCR hecho en 50 PDFs, 0 fallos`. El log imprime `s/doc`, los segundos efectivos por documento; con ese número se estima el total (sección 5). Para comparar configuraciones basta repetir el comando con otros `-Workers` y `-Jobs` (sección 4).

No conviene probar con `parte-sola.cmd -Limite 50`: el argumento se pasa a las tres tandas y cada una copiaría antes todos sus PDFs al disco local.

3.3. Paso 2: OCR de todo 2014

```
parte-sola.cmd
```

El script trabaja en tres tandas, una detrás de otra. En cada una copia sus PDFs al disco local, hace el OCR allí en paralelo, devuelve los PDFs al USB y borra la copia local. La copia local evita escribir miles de veces en el USB durante el OCR.

Tanda	Especialidades	PDFs	Tamaño
A	penal	3 586	7,1 GB
B	contencioso administrativo laboral y contencioso administrativo previsional	3 895	12,1 GB
C	revisión de procedimiento coactivo, contencioso administrativo, civil, laboral, constitucional, familia civil, comercial, familia tutelar y familia penal	3 992	11,8 GB

Espacio en disco. Se necesita en el disco local al menos el tamaño de la tanda más grande, **12,1 GB**, más un margen. El script comprueba el espacio libre antes de copiar y se detiene con un mensaje si no alcanza. En ese caso, usar otro disco:

```
parte-sola.cmd -Local D:\ocr
```

El USB tenía 21,8 GB libres al preparar el kit; el OCR reescribe los PDFs con un tamaño casi igual (un PDF de 6 páginas creció un 1,4 %), así que no debería agotarlo.

Vigilar los primeros minutos. El ciclo copiar, procesar y devolver no se ha ejecutado nunca de principio a fin. Conviene mirar la tanda A hasta que aparezca el primer `Devolviendo penal al USB`, antes de dejar la PC sola.

No desconectar el USB mientras se ve `Devolviendo ... al USB`. Al terminar todo aparece `TERMINADO`.

3.4. Si se corta

Un apagón o una ventana cerrada no estropean los PDFs: la escritura es atómica (archivo temporal y renombrado). Basta lanzar *exactamente* el mismo comando. Cada PDF que ya tiene texto se salta.

3.5. Paso 3: convertir a markdown

Con todo el OCR hecho:

```
ocr-lab.cmd -Convertir
```

Esto ejecuta `a_markdown.py` para 2014 con el manifiesto y el Excel del USB y deja el resultado en `X:\corpus-corte-suprema\markdown\`. El script detecta por fecha qué PDFs son más nuevos que su markdown y solo rehace esos.

4. Paralelismo

Con muchos núcleos rinde más procesar varios documentos a la vez que repartir las páginas de uno solo, porque la mayoría de los PDFs tiene pocas páginas (unas 7 a 10 de media en las muestras medidas). El OCR se parametriza con dos números:

Parámetro	Significado
`-Workers`	Documentos procesados a la vez. Por defecto: núcleos/2, máximo 16.
`-Jobs`	Hilos por documento. Por defecto: núcleos/`Workers`.

En una PC de 24 núcleos el valor automático es de 12 documentos a la vez con 2 hilos cada uno. Además, Tesseract se limita a un hilo por proceso (`OMP_THREAD_LIMIT=1`), para que los procesos no compitan entre sí, y la revisión previa de qué PDFs ya tienen texto también se hace en paralelo.

Cada documento en curso usa del orden de 0,5 a 1 GB de memoria. Si la PC se queda sin RAM, bajar `-Workers`. Los parámetros se añaden al final de cualquier comando:

```
parte-sola.cmd -Workers 12 -Jobs 2
parte-sola.cmd -Workers 6
parte-sola.cmd -Workers 1 -Jobs 24
```

La última línea es el modo anterior: un documento cada vez con todos los hilos. En la prueba de 50 conviene comparar el `s/doc` de dos configuraciones y quedarse con la mejor.

5. Cuánto tardará

Lo medido. En la PC de casa (4 núcleos), con un documento a la vez y 4 hilos, 50 PDFs de civil (520 páginas) tardaron unos 21 s por documento, es decir, unos 2,0 s por página. Con esa velocidad, los 11 338 pendientes serían unas 66 horas.

Lo que no se ha medido. En la máquina de 24 núcleos no se ha ejecutado nada. Si el rendimiento escalara linealmente con los núcleos, serían del orden de 11 horas, pero eso es una cota optimista: el disco, la memoria y el propio Tesseract no escalan perfectamente. Un intento de comparar en la PC de casa 4 documentos en paralelo contra uno solo salió peor (4,0 contra 2,0 s por página), pero se hizo con documentos distintos, sobre un equipo pequeño y con dos hilos reales, así que no permite concluir nada sobre el laboratorio. La cifra útil saldrá de la prueba con 50 PDFs en la propia máquina.

Revisión previa. Antes de empezar cada tanda, `ocr.py` recorre sus PDFs para ver cuáles ya tienen texto. En la PC de casa, con la versión secuencial, recorrer los 11 473 de 2014 tardó unos 30 minutos; ahora se hace en paralelo y debería ser bastante menos, pero no se ha medido sobre el conjunto completo.

6. Comprobar el resultado

- El log queda en `X:\corpus-corte-suprema\ocr-lab-2014-NOMBREPC.log` y cada tanda debe terminar con `0 fallos`.
- Si algún PDF falla, aparece en `X:\corpus-corte-suprema\pdfs\fallos-ocr.csv` con el motivo.

- `ocr.py` da por bueno un PDF solo si el resultado devuelve al menos 200 caracteres con `pdftotext`; si no, conserva el original.
- Para saber qué queda pendiente basta lanzar `ocr-lab.cmd`: recorre los PDFs del USB e imprime `necesitan OCR`; si es cero, imprime `nada que hacer`. Si no es cero, empieza a procesarlos directamente en el USB.

En la prueba de 50 PDFs del 4 de noviembre, los 50 quedaron con texto: el menor con 2 902 caracteres, la mediana con 21 168 y el mayor con 94 607. El texto es legible, pero en los encabezados el OCR deja frases desordenadas o con errores (por ejemplo, un membrete partido en columnas). Conviene revisar a mano unos cuantos markdown antes de usarlos para entrenar.

7. Después del OCR

1. `ocr-lab.cmd -Convertir` (arriba) genera el markdown de 2014.
2. Subir a Drive `pdfs/*/2014/` y `markdown/*/2014/`, y luego marcar el año como subido con `lote.py -subido 2014`, según el flujo descrito en `testeo/README.md`. Esa orden borra los PDFs y conserva el markdown.
3. Recordatorio: `estado.json` aún no marca 2014 como subido.

8. Cambios en el código

Se modificó `testeo/ocr.py`:

- `-output-type pdf`: no hace falta Ghostscript, que no se pudo instalar sin administrador. El PDF/A no se necesita para este corpus.
- Llama a `python -m ocrmypdf` en vez del ejecutable `ocrmypdf`, que solo existe con una instalación normal de pip. Así funciona con el Python portátil.
- Nuevo `-workers`: varios documentos en paralelo. Con el valor por defecto, 1, se comporta como antes.
- La revisión previa de qué PDFs tienen texto se hace con hilos.
- Se corrigió un error que ya existía: combinar `-especialidad` con `-anio` no encontraba ningún PDF, porque el patrón buscaba un nivel de carpetas de más.

9. Si el laboratorio no deja ejecutar los `.cmd`

Los `.cmd` son solo atajos: llaman a PowerShell y a Python. Hay tres niveles de respaldo, para probar en orden desde la misma consola en `X:\ocr-offline`.

Nivel 1: PowerShell funciona, el `.cmd` no.

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\ocr-lab.ps1 -Limite 50 -
    Especialidad civil
powershell -NoProfile -ExecutionPolicy Bypass -File .\ocr-parte.ps1 -Limpiar -
    Especialidad penal
powershell -NoProfile -ExecutionPolicy Bypass -File .\ocr-parte.ps1 -Limpiar -
    Especialidad contencioso-adm-laboral,contencioso-adm-previsional
powershell -NoProfile -ExecutionPolicy Bypass -File .\ocr-parte.ps1 -Limpiar -
    Especialidad revision-de-procedimiento-coactivo,contencioso-administrativo,civil,
    laboral,constitucional,familia-civil,comercial,familia-tutelar,familia-penal
powershell -NoProfile -ExecutionPolicy Bypass -File .\ocr-lab.ps1 -Convertir
```

Las tres líneas con `ocr-parte.ps1` son las tres tandas de `parte-sola.cmd`.

Nivel 2: PowerShell también está bloqueado. Se usa el Python portátil directamente, con el lanzador `ocr-lab.py`, que hace lo mismo sin `.cmd` ni `.ps1`:

```
portable\python\python.exe ocr-lab.py --simular --todo
portable\python\python.exe ocr-lab.py --limite 50 --especialidad civil
portable\python\python.exe ocr-lab.py --todo
portable\python\python.exe ocr-lab.py --convertir
```

`-simular` solo muestra los pasos sin ejecutarlos. Admite también `-local D:\ocr`, `-workers` y `-jobs`.

Nivel 3: no deja ejecutar nada desde el USB. Copiar `X:\ocr-offline` a `C:\ocr-offline` y repetir los niveles 1 o 2 desde allí; los datos se siguen leyendo de `X:`. Si tampoco funciona, la PC bloquea programas en carpetas de usuario por política del laboratorio. Eso no se resuelve desde aquí: hay que pedir al encargado que permita ejecutar Python y Tesseract desde el USB, o usar otra PC. No conviene intentar saltarse la política.

Estado de la prueba. `ocr-lab.py` compila y su modo `-simular` muestra los pasos correctos, pero el lanzador no se ha ejecutado con OCR real.

10. Qué no se ha probado

- El kit solo se ha ejecutado en la PC de casa. En el laboratorio puede haber bloqueos de seguridad que impidan ejecutar programas desde el USB. En ese caso, copiar `X:\ocr-offline` a `C:\ocr-offline` y lanzar desde allí; los datos se siguen leyendo de `X:`.
- El paralelismo con muchos núcleos y el ciclo completo de copiar, hacer OCR, devolver y borrar la copia local no se han ejecutado de principio a fin. Por eso la prueba de 50 va primero y la tanda A se vigila al inicio.
- Se asume Windows de 64 bits; los paquetes de Python incluidos son para Python 3.13.